Love2.c

```c

/*
#include <stdio.h>

// Function to find sum of digits of a number
int sumOfDigits(int n) {
    int sum = 0;
    while (n != 0) {
        sum += n % 10; // Add last digit to sum
        n = n / 10;    // Remove last digit
    }
    return sum;
}

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    int result = sumOfDigits(num);
    printf("Sum of digits of %d is %d\n", num, result);

    return 0;
}
*/


/*
#include <stdio.h>
#include <math.h>  // Needed for sqrt()

// Function to find square root of a number
double findSquareRoot(double n) {
    if (n < 0) {
        printf("Error: Negative number has no real square root.\n");
        return -1; // Error code
    }
    return sqrt(n); // Using math.h function
}

int main() {
    double num;
    printf("Enter a number: ");
    scanf("%lf", &num);

    double result = findSquareRoot(num);
    if (result != -1) {
        printf("Square root of %.2lf is %.2lf\n", num, result);
```

```c
    }

    return 0;
}
*/




/*
#include <stdio.h>

// Function to check temperature and print Hot or Cold
void checkTemperature(int temp) {
    if (temp >= 30) {
        printf("Hot\n");
    } else {
        printf("Cold\n");
    }
}

int main() {
    int temperature;
    printf("Enter the temperature in Celsius: ");
    scanf("%d", &temperature);

    checkTemperature(temperature);  // Call the function

    return 0;
}
*/



/*
#include <stdio.h>

// Function to calculate x^y
double myPow(double x, int y) {
    double result = 1.0;
    int power = y > 0 ? y : -y; // Use absolute value of exponent

    for (int i = 1; i <= power; i++) {
        result *= x;  // Multiply x y times
    }

    // If exponent is negative, return reciprocal
    if (y < 0) {
        return 1 / result;
    }
```

```c
    return result;
}

int main() {
    double base;
    int exponent;

    printf("Enter base: ");
    scanf("%lf", &base);
    printf("Enter exponent: ");
    scanf("%d", &exponent);

    double result = myPow(base, exponent);
    printf("%.2lf ^ %d = %.5lf\n", base, exponent, result);

    return 0;
}
*/



/*
#include <stdio.h>
#include <stdbool.h>

// Function to calculate factorial
long long factorial(int n) {
    long long fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
}

// Function to check prime
bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

// Function to check odd or even
void oddOrEven(int n) {
    if (n % 2 == 0)
        printf("%d is Even\n", n);
    else
        printf("%d is Odd\n", n);
}
```

```c

int main() {
    int choice, num;

    do {
        // Display menu
        printf("\n---- Menu ----\n");
        printf("1. Factorial of a number\n");
        printf("2. Prime or Not\n");
        printf("3. Odd or Even\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter a number to find factorial: ");
                scanf("%d", &num);
                if (num < 0)
                    printf("Factorial not defined for negative numbers.\n");
                else
                    printf("Factorial of %d = %lld\n", num, factorial(num));
                break;

            case 2:
                printf("Enter a number to check prime: ");
                scanf("%d", &num);
                if (isPrime(num))
                    printf("%d is a Prime Number\n", num);
                else
                    printf("%d is Not a Prime Number\n", num);
                break;

            case 3:
                printf("Enter a number to check Odd or Even: ");
                scanf("%d", &num);
                oddOrEven(num);
                break;

            case 4:
                printf("Exiting the program. Goodbye!\n");
                break;

            default:
                printf("Invalid choice. Please try again.\n");
        }

    } while(choice != 4);  // Repeat until user selects Exit

    return 0;
```

```c
}
*/



/*
#include <stdio.h>

int main() {
    int studentClass, failedSubjects, grace = 0;

    // Input class and failed subjects
    printf("Enter class obtained by student (1 for First, 2 for Second, 3 for Third): ");
    scanf("%d", &studentClass);
    printf("Enter number of subjects failed: ");
    scanf("%d", &failedSubjects);

    switch(studentClass) {
        case 1:  // First class
            if (failedSubjects > 3) {
                grace = 0;
            } else {
                grace = failedSubjects * 5;
            }
            break;

        case 2:  // Second class
            if (failedSubjects > 2) {
                grace = 0;
            } else {
                grace = failedSubjects * 4;
            }
            break;

        case 3:  // Third class
            if (failedSubjects > 1) {
                grace = 0;
            } else {
                grace = 5;
            }
            break;

        default:
            printf("Invalid class entered.\n");
            return 0; // Exit program
    }

    printf("Grace marks awarded: %d\n", grace);

    return 0;
```

```c
}
*/




/*
#include <stdio.h>

// Function to calculate factorial
long long factorial(int n) {
    if (n < 0) {
        return -1; // Error code for negative numbers
    }
    long long fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
}

int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);

    long long result = factorial(num);
    if (result == -1) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        printf("Factorial of %d is %lld\n", num, result);
    }

    return 0;
}
*/


/*
#include <stdio.h>

// Function to calculate a^b
double power(double a, int b) {
    double result = 1.0;
    int exp = b > 0 ? b : -b;  // Absolute value of exponent

    for (int i = 1; i <= exp; i++) {
        result *= a;
    }

```

```c
    if (b < 0) {
        return 1 / result;  // Handle negative exponent
    }
    return result;
}

int main() {
    double base;
    int exponent;

    printf("Enter base (a): ");
    scanf("%lf", &base);
    printf("Enter exponent (b): ");
    scanf("%d", &exponent);

    double result = power(base, exponent);
    printf("%.2lf ^ %d = %.5lf\n", base, exponent, result);

    return 0;
}
*/



/*
#include <stdio.h>
#include <string.h>

// Function to append Roman numeral to result string
void appendRoman(char *result, const char *roman) {
    strcat(result, roman);
}

// Function to convert number to Roman numeral
void yearToRoman(int year, char *roman) {
    // Arrays of values and corresponding Roman symbols
    int values[] = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
    char *symbols[] = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"};

    roman[0] = '\0'; // Initialize empty string

    for (int i = 0; i < 13; i++) {
        while (year >= values[i]) {
            appendRoman(roman, symbols[i]);
            year -= values[i];
        }
    }
}

```

```c
int main() {
    int year;
    char roman[50]; // Buffer to store Roman numeral

    printf("Enter a year: ");
    scanf("%d", &year);

    if (year <= 0) {
        printf("Year must be positive.\n");
        return 0;
    }

    yearToRoman(year, roman);
    printf("Roman equivalent of %d is %s\n", year, roman);

    return 0;
}
*/

// or

/*
#include <stdio.h>
#include <string.h>

void romanise(int y, int k, const char *ch) {
    int j = y / k;
    for (int i = 1; i <= j; i++) {
        printf("%s", ch);
    }
}

int main() {
    int yr;
    printf("Enter year: ");
    scanf("%d", &yr);

    romanise(yr, 1000, "m");
    yr %= 1000;
    romanise(yr, 900, "cm");
    yr %= 900;
    romanise(yr, 500, "d");
    yr %= 500;
    romanise(yr, 400, "cd");
    yr %= 400;
    romanise(yr, 100, "c");
    yr %= 100;
    romanise(yr, 90, "xc");
    yr %= 90;
    romanise(yr, 50, "l");
```

```c
    yr %= 50;
    romanise(yr, 40, "xl");
    yr %= 40;
    romanise(yr, 10, "x");
    yr %= 10;
    romanise(yr, 9, "ix");
    yr %= 9;
    romanise(yr, 5, "v");
    yr %= 5;
    romanise(yr, 4, "iv");
    yr %= 4;
    romanise(yr, 1, "i");

    return 0;
}
*/


/*
#include <stdio.h>

// Function to check leap year
int isLeapYear(int year) {
    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
        return 1; // Leap year
    else
        return 0; // Not leap year
}

int main() {
    int year;
    printf("Enter a year: ");
    scanf("%d", &year);

    if (isLeapYear(year))
        printf("%d is a Leap Year\n", year);
    else
        printf("%d is Not a Leap Year\n", year);

    return 0;
}
*/

/*
#include <stdio.h>

// Function to print prime factors
void primeFactors(int n) {
    int i;

```

```c
    printf("Prime factors are: ");

    for (i = 2; i <= n; i++) {
        while (n % i == 0) {
            printf("%d ", i);
            n = n / i;
        }
    }
}

int main() {
    int num;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num <= 0) {
        printf("Please enter a positive number.\n");
    } else {
        primeFactors(num);
    }

    return 0;
}
*/









/*
#include <stdio.h>
#include <math.h>

// Function to calculate sum, average and standard deviation
void stats(int *sum, int *avg, double *stdev) {
    int n1, n2, n3, n4, n5;

    printf("Enter 5 numbers: ");
    scanf("%d%d%d%d%d", &n1, &n2, &n3, &n4, &n5);

    *sum = n1 + n2 + n3 + n4 + n5;
    *avg = *sum / 5;

    *stdev = sqrt(
        (pow((n1 - *avg), 2) +
         pow((n2 - *avg), 2) +
         pow((n3 - *avg), 2) +
```

```c
        pow((n4 - *avg), 2) +
        pow((n5 - *avg), 2)) / 4.0
    );
}

int main() {
    int sum, avg;
    double stdev;

    stats(&sum, &avg, &stdev);

    printf("\nSum = %d", sum);
    printf("\nAverage = %d", avg);
    printf("\nStandard Deviation = %lf\n", stdev);

    return 0;
}
*/



/*
#include <stdio.h>

// Function to calculate power and factorial
void power_fact(float x, int y, int num, float *power, int *fact) {
    float res = 1;
    int i;

    // Calculate power (x^y)
    for (i = 1; i <= y; i++) {
        res = res * x;
    }
    *power = res;

    // Calculate factorial
    res = 1;
    for (i = 1; i <= num; i++) {
        res = res * i;
    }
    *fact = (int)res;
}

int main() {
    float a, pow;
    int b, number, factorial;

    printf("Enter a and b for calculating a raised to b: ");
    scanf("%f %d", &a, &b);

```

```c
    printf("Enter number whose factorial is to be calculated: ");
    scanf("%d", &number);

    power_fact(a, b, number, &pow, &factorial);

    printf("Power = %f\nFactorial = %d\n", pow, factorial);

    return 0;
}
*/



/*
#include <stdio.h>

int main() {
    float x = 3.14;
    float *y;
    float **z;

    y = &x;   // y stores address of x
    z = &y;   // z stores address of y

    // Print addresses
    printf("%p %p %p\n", &x, &y, &z);

    // Print stored addresses
    printf("%p %p\n", y, z);

    // Print values
    printf("%f %f %f\n", x, *y, **z);

    return 0;
}
*/



/*
#include <stdio.h>

// Function to circularly shift values
void circularShift(int *x, int *y, int *z) {
    int temp = *z;

    *z = *y;
    *y = *x;
    *x = temp;
}
```

```c

int main() {
    int a = 5, b = 8, c = 10;

    circularShift(&a, &b, &c);

    printf("After shift:\n");
    printf("a = %d b = %d c = %d\n", a, b, c);

    return 0;
}
*/




/*
#include <stdio.h>

// Function to convert weight
void convertWeight(float kg, float *g, float *t, float *p) {
    *g = kg * 1000;
    *t = kg * 0.001;
    *p = kg * 2.20462;
}

int main() {
    float kg, grams, tons, pounds;

    printf("Enter weight in kg: ");
    scanf("%f", &kg);

    convertWeight(kg, &grams, &tons, &pounds);

    printf("Grams = %.2f\n", grams);
    printf("Tons = %.4f\n", tons);
    printf("Pounds = %.2f\n", pounds);

    return 0;
}
*/




/*
#include <stdio.h>
#include <math.h>

// Distance between two points
```

```c
float distance(float x1, float y1, float x2, float y2) {
    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

// Area of triangle using 3 points
float area(float x1, float y1, float x2, float y2, float x3, float y3) {
    float a = distance(x1, y1, x2, y2);
    float b = distance(x2, y2, x3, y3);
    float c = distance(x3, y3, x1, y1);

    float s = (a + b + c) / 2;

    return sqrt(s * (s - a) * (s - b) * (s - c));
}

// Check if point lies inside triangle
int isInside(float x1, float y1, float x2, float y2, float x3, float y3, float x, float y) {
    float A = area(x1, y1, x2, y2, x3, y3);
    float A1 = area(x, y, x2, y2, x3, y3);
    float A2 = area(x1, y1, x, y, x3, y3);
    float A3 = area(x1, y1, x2, y2, x, y);

    if (fabs(A - (A1 + A2 + A3)) < 0.0001)
        return 1;
    else
        return 0;
}

int main() {
    float x1=0,y1=0, x2=5,y2=0, x3=0,y3=5;
    float x=2, y=2;

    if (isInside(x1,y1,x2,y2,x3,y3,x,y))
        printf("Point is inside triangle\n");
    else
        printf("Point is outside triangle\n");

    return 0;
}
*/




/*
#include <stdio.h>

// Recursive function to calculate sum of digits
int rsum(int n) {
    if (n == 0)
```

```c
        return 0;
    else
        return (n % 10) + rsum(n / 10);
}

int main() {
    int num, sum;

    printf("Enter a 5-digit number: ");
    scanf("%d", &num);

    sum = rsum(num);

    printf("Sum of digits is %d\n", sum);

    return 0;
}
*/



/*
#include <stdio.h>

// Recursive function to print prime factors
void factorize(int n, int i) {
    if (n == 1)
        return;

    if (n % i == 0) {
        printf("%d ", i);
        factorize(n / i, i);  // divide and continue with same i
    } else {
        factorize(n, i + 1);  // try next number
    }
}

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    printf("Prime factors are: ");
    factorize(num, 2);

    return 0;
}
*/

```

```c
/*
#include <stdio.h>

// Recursive function for Fibonacci
int fibo(int n) {
    if (n == 0 || n == 1)
        return n;
    else
        return fibo(n - 1) + fibo(n - 2);
}

int main() {
    int terms = 25, i;

    for (i = 0; i < terms; i++) {
        printf("%d\t", fibo(i));
    }

    return 0;
}
*/



/*
#include <stdio.h>

void binary(int n) {
    int bin[32], i = 0;

    while (n > 0) {
        bin[i++] = n % 2;
        n = n / 2;
    }

    printf("Binary = ");
    for (int j = i - 1; j >= 0; j--) {
        printf("%d", bin[j]);
    }
}

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    binary(num);
```

```c
    return 0;
}
*/




/*
#include <stdio.h>

void binary(int n) {
    if (n == 0)
        return;

    binary(n / 2);
    printf("%d", n % 2);
}

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    printf("Binary = ");
    binary(num);

    return 0;
}
*/




/*
#include <stdio.h>

int sum(int n) {
    if (n == 1)
        return 1;
    else
        return n + sum(n - 1);
}

int main() {
    int result = sum(25);
    printf("Sum = %d\n", result);
    return 0;
}
*/

```

```c
/*
#include <stdio.h>

// Recursive function for Tower of Hanoi
void hanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", from, to);
        return;
    }

    hanoi(n - 1, from, aux, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    hanoi(n - 1, aux, to, from);
}

int main() {
    int n = 4;

    printf("Steps to solve Tower of Hanoi:\n");
    hanoi(n, 'A', 'C', 'B');

    return 0;
}
*/




/*
#include <stdio.h>

// Function to find maximum using pointers
int max(int *a, int *b) {
    if (*a > *b)
        return *a;
    else
        return *b;
}

int main() {
    int x, y;

    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);

    int result = max(&x, &y);

```

```c
    printf("Maximum number = %d\n", result);

    return 0;
}
*/



/*
#include <stdio.h>

int main() {
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Array in reverse order:\n");
    for (i = n - 1; i >= 0; i--) {
        printf("%d ", arr[i]);
    }

    return 0;
}
*/



/*
#include <stdio.h>

int main() {
    char ch = 'A';
    char *ptr = &ch;

    printf("Alphabets are:\n");

    for (; *ptr <= 'Z'; (*ptr)++) {
        printf("%c ", *ptr);
    }

    return 0;
}
*/
```

```c



/*

*/
```